MACHINE VISION ASSIGNMENT 2

Sreehitha Korrapati-6/10/2026
KSU ID: 001228797

ABSTRACT:
In this assignment we work on log transformation, power-law transformation, and histogram equalization and implemented them in MATLAB to enhance grayscale images. First, log and power-law transformations are applied to a Fourier spectrum image to improve the visibility of dark regions. Histogram equalization is then applied to a low-contrast grayscale image to enhance its contrast. The results also showed that both log and power-law transformations increased the brightness of darker pixels, making image details easier to observe. Histogram equalization improved the overall contrast by redistributing pixel intensities over a wider range. Mean and standard deviation values are also calculated before and after histogram equalization to evaluate the enhancement.

TEST RESULTS:

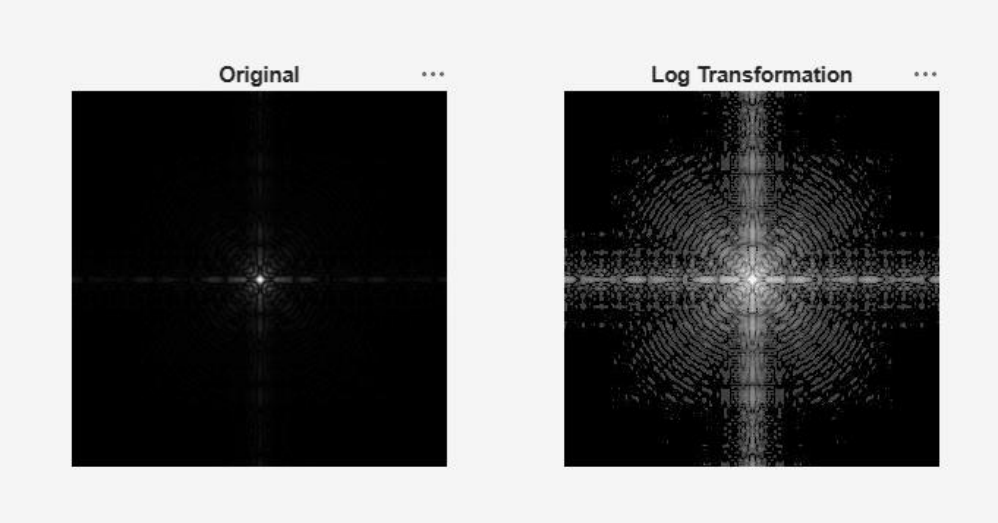

Fig1:Log Transformation: The original Fourier spectrum image is very dark with most pixel values near zero. After applying log transformation ($s = c \cdot \log(1+r)$, $c = 55.49$), the dark regions are brightened and frequency details that were invisible in the original become clearly visible.

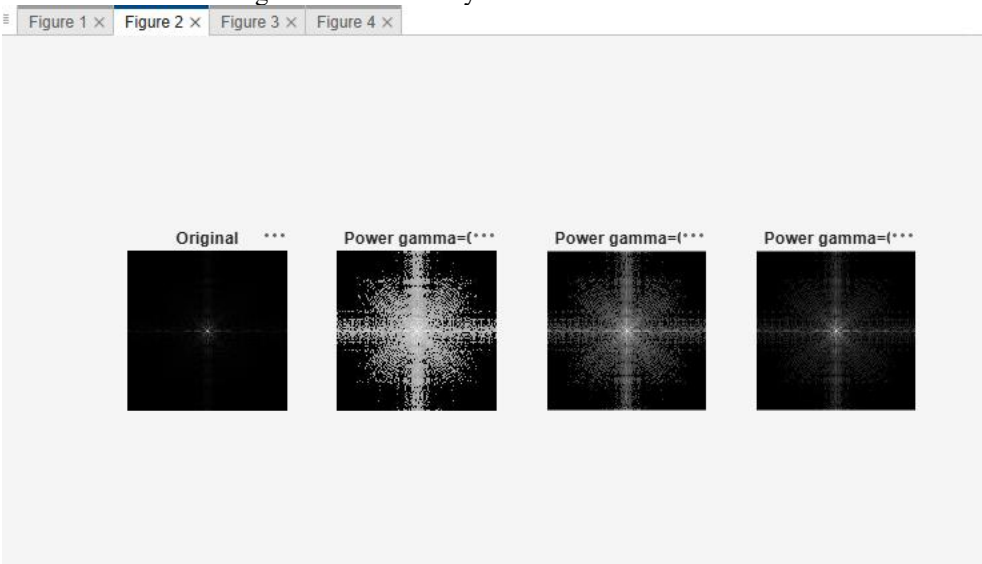

Fig2: Power-Law Transformation: This was applied with three gamma values less than 1. At $\gamma = 0.1$ the image is heavily brightened and looks similar to the log result. As gamma increases to 0.3 and 0.5 the brightening effect becomes weaker and more detail is preserved in the brighter regions.

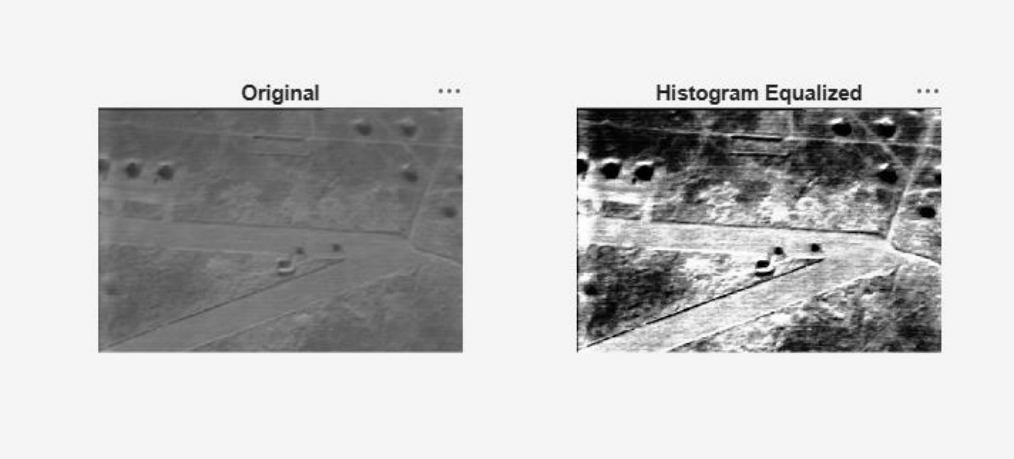

Fig 3 : Histogram Equalization: The original image has low contrast with all pixel values concentrated in a narrow mid-range. After histogram equalization the intensity values are spread across the full 0–255 range, making dark and bright regions much more distinguishable.

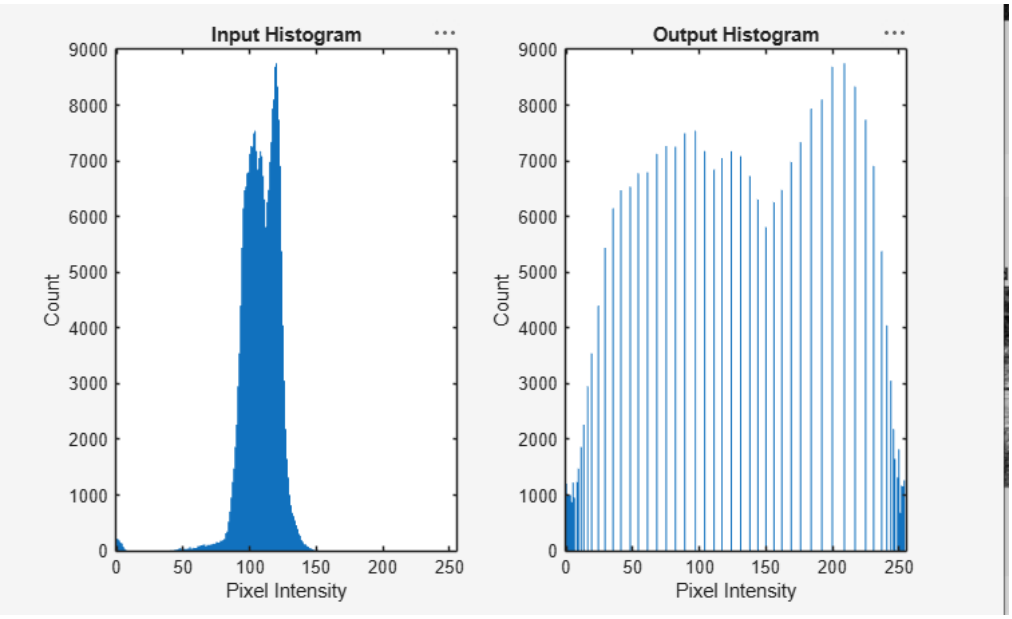

Fig4: Histograms: The input histogram shows pixel values tightly clustered between 90 and 140, which explained the low contrast in the original image. After equalization the output histogram is kind of spread across the full range, and the standard deviation increased from 13.8 to 73.9, confirmed the contrast improvement.

DISCUSSION:

This assignment helped me understand how image enhancement techniques work and how they affect image quality. The log and power-law transformations both improved the visibility of darker regions in the image, but the power-law transformation was more flexible because different gamma values produced different enhancement effects. Histogram equalization was effective for improving the contrast of low-contrast images by spreading pixel values over a wider intensity range. Implementing these techniques in MATLAB helped me better understand how pixel values are modified during image enhancement. There was one challenge with selecting appropriate parameters and comparing the results of different methods. Finally in the future, I want to try more advanced methods like adaptive histogram equalization to achieve better local contrast enhancement.

CODE:

```
clear;
```

```
clc;
close all;

%% Log transformation

img = imread('fourierspectrum.pgm');

[r,c] = size(img);

c_log = 255 / log(1 + double(max(img(:))));

output = zeros(r,c,'uint8');

for i = 1:r
   for j = 1:c

      pixel = double(img(i,j));

      output(i,j) = uint8(c_log * log(1 + pixel));

   end
end

imwrite(output,'log_output.jpg');

figure;

subplot(1,2,1);
imshow(img);
title('Original');

subplot(1,2,2);
imshow(imread('log_output.jpg'));
title('Log Transformation');

saveas(gcf,'log_comparison.png');

%% Power-law transformation (gamma = 0.1)

output = zeros(r,c,'uint8');

c_pow = 255 / (255 ^ 0.1);

for i = 1:r
   for j = 1:c

      pixel = double(img(i,j));

      output(i,j) = uint8(c_pow * (pixel ^ 0.1));

   end
end

imwrite(output,'power_gamma0.1.jpg');

%% Power-law transformation (gamma = 0.3)
```

```
output = zeros(r,c,'uint8');

c_pow = 255 / (255 ^ 0.3);

for i = 1:r
   for j = 1:c

      pixel = double(img(i,j));

      output(i,j) = uint8(c_pow * (pixel ^ 0.3));

   end
end

imwrite(output,'power_gamma0.3.jpg');

%% Power-law transformation (gamma = 0.5)

output = zeros(r,c,'uint8');

c_pow = 255 / (255 ^ 0.5);

for i = 1:r
   for j = 1:c

      pixel = double(img(i,j));

      output(i,j) = uint8(c_pow * (pixel ^ 0.5));

   end
end

imwrite(output,'power_gamma0.5.jpg');

figure;

subplot(1,4,1);
imshow(img);
title('Original');

subplot(1,4,2);
imshow(imread('power_gamma0.1.jpg'));
title('Power gamma=0.1');

subplot(1,4,3);
imshow(imread('power_gamma0.3.jpg'));
title('Power gamma=0.3');

subplot(1,4,4);
imshow(imread('power_gamma0.5.jpg'));
title('Power gamma=0.5');

saveas(gcf,'power_comparison.png');

%% Histogram equalization
```

```
img2 = imread('banker.jpeg');

[r2,c2] = size(img2);

N = r2 * c2;

hist_in = zeros(1,256);

for i = 1:r2
   for j = 1:c2

      val = img2(i,j) + 1;

      hist_in(val) = hist_in(val) + 1;

   end
end

cdf = zeros(1,256);
cdf(1) = hist_in(1);

for k = 2:256

   cdf(k) = cdf(k-1) + hist_in(k);

end

cdf_min = 0;

for k = 1:256

   if cdf(k) > 0
      cdf_min = cdf(k);
      break;
   end

end

lut = zeros(1,256,'uint8');

for v = 1:256

   lut(v) = uint8(floor(((cdf(v) - cdf_min) / (N - cdf_min)) * 255));

end

output2 = zeros(r2,c2,'uint8');

for i = 1:r2
   for j = 1:c2

      output2(i,j) = lut(img2(i,j) + 1);

   end
end
```

```
imwrite(output2,'banker_equalized.jpg');

figure;

subplot(1,2,1);
imshow(img2);
title('Original');

subplot(1,2,2);
imshow(imread('banker_equalized.jpg'));
title('Histogram Equalized');

saveas(gcf,'heq_comparison.png');
hist_out = zeros(1,256);

for i = 1:r2
   for j = 1:c2

      val = output2(i,j) + 1;

      hist_out(val) = hist_out(val) + 1;

   end
end

figure;

subplot(1,2,1);
bar(0:255, hist_in, 1);
title('Input Histogram');
xlabel('Pixel Intensity');
ylabel('Count');

subplot(1,2,2);
bar(0:255, hist_out, 1);
title('Output Histogram');
xlabel('Pixel Intensity');
ylabel('Count');

saveas(gcf,'histogram_comparison.png');

mean_in = 0;

for i = 1:r2
   for j = 1:c2

      mean_in = mean_in + double(img2(i,j));

   end
end

mean_in = mean_in / N;

mean_out = 0;
```

```
for i = 1:r2
  for j = 1:c2

    mean_out = mean_out + double(output2(i,j));

  end
end

mean_out = mean_out / N;

std_in = 0;

for i = 1:r2
  for j = 1:c2

    std_in = std_in + (double(img2(i,j)) - mean_in)^2;

  end
end

std_in = sqrt(std_in / N);

std_out = 0;

for i = 1:r2
  for j = 1:c2

    std_out = std_out + (double(output2(i,j)) - mean_out)^2;

  end
end

std_out = sqrt(std_out / N);

fprintf('Input  - Mean: %.2f, Std: %.2f\n', mean_in, std_in);
fprintf('Output - Mean: %.2f, Std: %.2f\n', mean_out, std_out);
```